

claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_7049e1ed-b93\agent-a3d602f49b4362489.jsonl`
- SHA-256: `3d1fd01d86f087230ceba98e9b7b4859f4f1e5e48a6ed3e530273bb8199b1e22`
- Source modified: `2026-07-02T04:59:03+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_7049e1ed-b93`
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`
```

Transcript

1. user (2026-07-02T04:54:07.888Z)

You are an adversarial code reviewer. Today is 2026-07-02. Review PR #416 in the repo at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer (gh pr view 416, gh pr diff 416; the working copy is in worktree C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-gdrive-api-safety ? read code there for context).

The PR claims to fix: R1-16 (gdrive_api RAM buffering + unsafe put ordering) (evidence in C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/AUDIT_REPORT_khelsutra-guru_2026-07-02.md ? Grep for the IDs; honor the verifier notes' scoping corrections). Hunt REAL defects: correctness bugs in the diff, broken/vacuous tests, behavior changes beyond the brief, staging accidents (files that should not be in the diff ? check the file list!), convention violations (default-ON when brief says OFF, missing schema-version check, CWD assumptions), and whether the fix actually addresses the finding. READ-ONLY ? do not modify or comment on GitHub. blocker = must not merge; major = fix before merge; minor = nice-to-have.

2. user (2026-07-02T04:54:16.804Z)

```
title: fix(storage): gdrive-api ? chunked streaming download + loss-proof idempotent put
state: OPEN
author: avidullu (Avi)
labels:
assignees:
reviewers:
projects:
milestone:
number: 416
url: https://github.com/Khelsutra/badminton-highlight-indexer/pull/416
additions: 199
deletions: 21
auto-merge: disabled
--
```

What / why

Fixes audit finding ****R1-18**** (AUDIT_REPORT_khelsutra-guru_2026-07-02) on ``backend/storage/gdrive_api.py`` ? two latent hazards on the Drive-API storage hot path before the reel round-trip goes live:

1. ****Whole-file RAM buffering on download.**** ``fetch_to_local`` did ``get_media().execute()``, materializing the entire object as one ``bytes`` value ? an OOM waiting to happen for a multi-GB source/reel on a memory-limited Cloud Run box. It now streams via ``googleapiclient.http.MediaIoBaseDownload`` in bounded ****32 MiB chunks**** (lazy import behind a ``_downloader`` seam, mirroring the existing ``_media_body`` seam/convention). The stream writes to a ``dst + ".part"`` sibling and ``os.replace``s into place, so a mid-download failure can never

leave a truncated `dst` that a later `overwrite=False` call would trust as complete.

2. **Delete-before-upload data-loss window in `put()`**. A re-put deleted the existing file at the path and only then ran `files().create(...)` ? if the resumable create failed (network/quota), the reel was gone at both ends. Per the audit verifier's refinement, re-puts now overwrite **in place** via `files().update(fileId=existing, media\_body=...)`: the old bytes survive until the new upload succeeds, the Drive file id is preserved across re-puts, and no duplicate-`(name, parent)` window opens (which a create-first-then-delete ordering would, given path resolution returns the first match among duplicates). Fresh paths still `files().create(...)`. The **idempotent-put** contract from PR #338 (exactly one file per path, latest bytes win) is preserved and still covered by the pre-existing test.

No config knobs, no behavior change for callers: same `StorageBackend` interface, same idempotence semantics ? only the failure modes improve.

Verification

Ran locally in an isolated worktree on top of `origin/master` (e410136):

```
- `python -m pytest tests/test_storage_gdrive_api.py -q` ? **14 passed** (was 10; 4 new tests:
`test_fetch_large_file_streams_in_chunks` (50-byte file / 4-byte fake chunks ? 13 `next_chunk()`
calls, byte-identical reassembly), `test_fetch_failure_mid_download_leaves_no_truncated_dst`,
`test_reput_updates_in_place_preserving_file_id`,
`test_put_failure_mid_upload_keeps_old_object_intact` (old bytes readable after a simulated mid-
upload failure; retry succeeds with exactly one file)).
- `python -m pytest tests -q -k storage` ? **57 passed, 2 skipped** (the full storage-backend
surface).
- `ruff check` + `ruff format --check` clean on both changed files; `mypy
backend/storage/gdrive_api.py` clean.
```

Not verifiable locally: the real `googleapiclient` path (`MediaIoBaseDownload` / `MediaFileUpload` / real Drive) ? the library is deliberately absent from the test env; the fakes drive the identical `next\_chunk()` loop and `files().update`/`create` call shapes. Full suite + coverage floor is CI's job (`run\_all\_tests.py` lane).

? Generated with [Claude Code](https://claude.com/claude-code)

3. user (2026-07-02T04:54:17.482Z)

172-[Omitted long context line]

173-- why it matters: DOC_STATUS is by convention the archival/lifecycle board and README is the ramp-up index; both are read at every session start. When they contradict the per-doc source-of-truth trackers, sessions either relitigate finished work or trust stale gating claims. The cleanup audit's own DoD (line 225) is blocked on exactly this README update.

174-- proposed fix: One docs-only reconcile PR: refresh DOC_STATUS.md:47 row (Phase 2 nearly done, remaining = P10 + P16+), add a DECISION_SNAPSHOT_REGRESSION row, fix docs/README.md:41-42 blurbs, and rewrite the NEXT_STEPS.md:6 'Where we are' paragraph to the post-06-21 state.

175-[Omitted long context line]

176-[Omitted long context line]

177-

178-### R1-15 [P2/M] test-gap ? badminton-highlight-indexer

179-****M4 quality-floor gate still missing ? the one unblocked item keeping the golden-fixtures track from DONE****

180-- anchor: `docs/GOLDEN\_REGRESSION\_FIXTURES.md:156` · tracked in:

docs/GOLDEN_REGRESSION_FIXTURES.md §7 M4; docs/BACKLOG.md:60 · finder: engine:audit-doc-reconcile

181-- evidence: GOLDEN_REGRESSION_FIXTURES.md:156 (M4 row) is ?; verified

tests/test_golden_quality_floor.py and eval_baselines/regression_baseline.json do not exist (ls: No such file). §9 DoD (line 197) requires M4 before the track can flip DONE. BACKLOG.md:60 calls it 'recommended-next; unblocked... the cheapest step to flip the track to DONE' (advice standing since ~06-16).

182-- why it matters: Today the golden suite is characterization-only: it catches behavior CHANGES but has no committed F1 floor or windowing fingerprint, so a deliberate re-freeze after a SERVED retune could silently lock in worse quality. It is also the sole unblocked blocker to

archiving the whole fixtures project.

183-- proposed fix: Implement M4 per the doc's own recipe: seed per-slug floors from the F1 already frozen in each fx_*/expected.json score (floor = current - tolerance), record a windowing fingerprint in eval_baselines/regression_baseline.json, keep CONTROL-only, add to the golden-crossos CI matrix.

184-[Omitted long context line]

185-[Omitted long context line]

186-

187-### R1-16 [P2/L] tech-debt ? badminton-highlight-indexer

188-****Decision-snapshot regression track: decisions locked 06-25, zero code shipped since; tracker P0 row also stale****

189-- anchor: `docs/DECISION\_SNAPSHOT\_REGRESSION.md:66` · tracked in:

docs/DECISION_SNAPSHOT_REGRESSION.md §7 (P1-P8) · finder: engine:audit-doc-reconcile

190-- evidence: §7 tracker (lines 63-73): P1-P8 all ?. Verified in code: grep

'build_stitch_plan' across backend/ and tests/ returns nothing (P2's pure stitch-plan seam was never factored out of backend/pipeline/stitcher/compiler.py), and tests/test_golden_fixtures.py has no stitch-plan snapshot. Minor rot: the P0 row (line 65) still shows ? '(this PR)' though the doc merged as #383 (commit 1283e18, 2026-06-25) and the header still says 'DRAFT'.

191-- why it matters: This gate exists precisely to bless genuine (non-AST-identical) refactors as quality-neutral ? i.e., exactly the class of change as the stalled #390 main.py split. Executing P1-P4 first would de-risk redoing the god-module split; leaving it a week-stale locked-decisions doc means refactors continue with no behavioral proof.

192-- proposed fix: Start §7 P1+P2 (canonical decision-snapshot schema + factor pure build_stitch_plan() out of compiler.py) as the next Claude-doable engine work; flip the P0 row to ? #383 and status to IN PROGRESS in the same PR. Ideally land P1-P4 before re-cutting the main.py split.

193-[Omitted long context line]

194-[Omitted long context line]

195-

196-### R1-17 [P2/M] tech-debt ? badminton-highlight-indexer

197-****Fetched-input scratch copies are never reclaimed anywhere (process jobs, compile jobs, /api/assets, cloud_run marker reads) ? 'OS temp reclamation' premise is false on Windows****

198-- anchor: `backend/storage/\_\_init\_\_.py:89` · tracked in: ? · finder: engine:correctness

199-[Omitted long context line]

200-- why it matters: Slow-burn disk exhaustion on exactly the appliance the product ships as; also inflates the free-space guard's false rejections.

201-- proposed fix: Wrap remote-input acquisition in a per-job scope that deletes the scratch dir in a finally (worker already owns the job lifecycle), or route scratch under output_base(cfg)/tmp with an age-based sweep at startup; make cloud_run._read_json use TemporaryDirectory.

202-[Omitted long context line]

4. user (2026-07-02T04:54:27.558Z)

187-### R1-16 [P2/L] tech-debt ? badminton-highlight-indexer

188-****Decision-snapshot regression track: decisions locked 06-25, zero code shipped since; tracker P0 row also stale****

189-- anchor: `docs/DECISION\_SNAPSHOT\_REGRESSION.md:66` · tracked in:

docs/DECISION_SNAPSHOT_REGRESSION.md §7 (P1-P8) · finder: engine:audit-doc-reconcile

190-- evidence: §7 tracker (lines 63-73): P1-P8 all ?. Verified in code: grep

'build_stitch_plan' across backend/ and tests/ returns nothing (P2's pure stitch-plan seam was never factored out of backend/pipeline/stitcher/compiler.py), and tests/test_golden_fixtures.py has no stitch-plan snapshot. Minor rot: the P0 row (line 65) still shows ? '(this PR)' though the doc merged as #383 (commit 1283e18, 2026-06-25) and the header still says 'DRAFT'.

191-- why it matters: This gate exists precisely to bless genuine (non-AST-identical) refactors as quality-neutral ? i.e., exactly the class of change as the stalled #390 main.py split. Executing P1-P4 first would de-risk redoing the god-module split; leaving it a week-stale locked-decisions doc means refactors continue with no behavioral proof.

192-- proposed fix: Start §7 P1+P2 (canonical decision-snapshot schema + factor pure build_stitch_plan() out of compiler.py) as the next Claude-doable engine work; flip the P0 row to ? #383 and status to IN PROGRESS in the same PR. Ideally land P1-P4 before re-cutting the main.py split.

193-[Omitted long context line]

194-[Omitted long context line]

195-

196-### R1-17 [P2/M] tech-debt ? badminton-highlight-indexer
197-**Fetched-input scratch copies are never reclaimed anywhere (process jobs, compile jobs, /api/assets, cloud_run marker reads) ? 'OS temp reclamation' premise is false on Windows**
198-- anchor: `backend/storage/\_\_init\_\_.py:89` · tracked in: ? · finder: engine:correctness
199-[Omitted long context line]
200-- why it matters: Slow-burn disk exhaustion on exactly the appliance the product ships as; also inflates the free-space guard's false rejections.
201-- proposed fix: Wrap remote-input acquisition in a per-job scope that deletes the scratch dir in a finally (worker already owns the job lifecycle), or route scratch under output_base(cfg)/tmp with an age-based sweep at startup; make cloud_run._read_json use TemporaryDirectory.
202-[Omitted long context line]
203-[Omitted long context line]
204-
205-### R1-18 [P2/S] critical-fix ? badminton-highlight-indexer
206-**GDriveApiStorage.fetch_to_local buffers the entire file in RAM (get_media().execute()); put() deletes the old object before the new upload succeeds**
207-- anchor: `backend/storage/gdrive\_api.py:144` · tracked in: ? · finder: engine:correctness
208-- evidence: gdrive_api.py:143-148: `data = self.\_service.files().get\_media(fileId=file\_id).execute()` then `fh.write(data)` ? the whole object is materialized as one bytes value (no MediaIoBaseDownload streaming).
gdrive_api.py:163-170: put() resolves an existing file at the path and `files().delete(...)` BEFORE `files().create(...)` ? delete-then-create is not atomic.
209-- why it matters: Latent OOM + data-loss on the Drive-API storage hot path before it goes live for the reel round-trip.
210-- proposed fix: Stream downloads with googleapiclient.http.MediaIoBaseDownload in chunks; for put(), create the new file first (or upload to a temp name) and delete the old one only after success.
211-[Omitted long context line]
212-[Omitted long matching line]
213-
214-### R1-19 [P2/S] critical-fix ? badminton-highlight-indexer
215-**config.json output.output_dir is silently clobbered by the CLI --output-dir DEFAULT on every server/CLI start ? the typed knob is dead, and the DB path moves with it**
216-- anchor: `backend/main.py:1992` · tracked in: ? · finder: engine:correctness
217-- evidence: main.py:1837-1841 defines `--output-dir` with default='output'; main.py:1992 unconditionally does `global\_config.output.output\_dir = resolve\_output\_dir(args.output\_dir)` ? the override fires even when the flag was never passed, so a config.json value can never take effect. models.py:274-276 documents the knob as if it's honored ('The CLI's --output-dir overrides this at startup' ? implying only when given). db_path derives from it (main.py:1995-1997).
218-- why it matters: A documented config knob that is a no-op on the main entry path, with data-location consequences (perceived data loss / split state).
219-- proposed fix: Change argparse default to None and only override when args.output_dir is not None (falling back to the config value through resolve_output_dir).
220-[Omitted long context line]
221-[Omitted long context line]
222-
223-### R1-20 [P2/S] critical-fix ? badminton-highlight-indexer
224-**upload_dir default 'output/uploads' is CWD-relative and never app-home-resolved ? packaged --app writes uploads into an arbitrary CWD (or 500s on an unwritable one) while DB/reels live under RALLY_APP_HOME**
225-- anchor: `backend/api/uploads.py:50` · tracked in: ? · finder: engine:correctness
226-- evidence: uploads.py:50 `upload\_dir = getattr(sec, 'upload\_dir', None) or 'output/uploads'` used verbatim at :89 (os.makedirs), :95, :170 ? grep confirms no resolve_output_dir/app_path is ever applied to it (only output.output_dir is resolved, main.py:1992 via app_home.py:83-100). models.py:336 default 'output/uploads'.
227-- why it matters: Breaks or misplaces the primary ingest path of the packaged app; exactly the class of CWD-vs-app-home bug P0a was built to eliminate (app_home.py:1-8).
228-- proposed fix: Resolve upload_dir through resolve_output_dir()/app_path() in _upload_cfg (relative ? app-home-rooted; absolute passes through), mirroring output_dir.
229-[Omitted long context line]
230-[Omitted long context line]
--
465-[Omitted long context line]

466-[Omitted long context line]
467-
468-### R2-3 [P2/S] ? sports-data-collector
469-Collector pins obsreport as a plain PyPI requirement while the name is unclaimed on PyPI ?
dependency-confusion window and install broken today**
470-- verdict: CONFIRMED
471-[Omitted long context line]
472-[Omitted long context line]
473-
474-### R2-4 [P2/S] ? badminton-highlight-indexer
475-No timeout on any serving-path ffmpeg subprocess ? one hung ffmpeg permanently wedges the
single-worker job queue**
476-- verdict: CONFIRMED
477-[Omitted long context line]
478-[Omitted long context line]
479-
480-### R2-5 [P1/S] ? badminton-highlight-indexer
481-Engine CLAUDE.md 'Current state' section is ~3 weeks false ? claims platform/owned-model
work 'has not started'**
482-- verdict: CONFIRMED
483-[Omitted long matching line]
484-[Omitted long context line]
485-
486-### R2-6 [P1/M] ? sports-data-collector
487-Golden-corpus registry rot: all 6 owned 'curated' videos point at a deleted absolute
local_path and the 4.9GB source videos are single-copy with no documented backup**
488-- verdict: CONFIRMED
489-[Omitted long context line]
490-[Omitted long context line]
491-
492-### R2-7 [P2/S] ? non-git
493-Roora vendor pilot record (paid deliverables + corrected golden labels + 3 videos) is a
single unversioned local copy at risk of loss**
494-- verdict: CONFIRMED
495-[Omitted long context line]
496-[Omitted long context line]
497-
498-### R2-8 [P1/S] ? badminton-highlight-indexer
499-Upload retention sweep tool exists but nothing schedules it ? unbounded upload growth +
unenforced 7-day privacy retention on any hosted deployment (#301)**
500-- verdict: CONFIRMED
501-[Omitted long context line]

5. user (2026-07-02T04:54:36.093Z)

203 - verifier reasoning: Personally verified in the current tree. (1)
backend/storage/__init__.py:80-82 docstring says the scratch copy "is left under the system temp
dir ? and relies on OS temp reclamation"; line 89 does tempfile.mkdtemp(prefix="rally-input-")
and line 90 fetches into it ? no deletion. (2) Every claimed call site leaks:
_execute_processing main.py:1168-1171 (no rmtree anywhere in main.py ? grep confirmed rmtree
exists only in eval/golden_real_fixtures.py, pipeline/stitcher/compiler.py,
pipeline/detectors/*); _execute_compile main.py:1683-1686; create_asset main.py:791-799 fetches
a full video copy just to MD5-hash it (main.py:805) then returns at 865-875 without deleting
local_working; get_source main.py:646 fetches per-request and serves it via FileResponse (line
660) ? a per-view full-video leak for remote sources. Plus a fifth site the finder missed: CLI
single-shot main.py:2056. cloud_run.py:219-222 _read_json mkdtemps "cr-read-" per marker/report
read, no cleanup. (3) No janitor: tools/cleanup_caches.py scans only WSL ~/clips, the output/
tree, and security.upload_dir (docstring lines 3-5, classify() lines 52-97) ? never
tempfile.gettempdir(); its "tmp*" rule (line 79-80) applies only inside those trees. (4) Not
fixed: git log for storage/__init__.py tops out at b6214e5 (repo-wide ruff format); gh issue 301
confirmed to cover only the T11 upload-retention scheduled job, not temp scratch. (5) Guard
interplay real: require_free_bytes(tempfile.gettempdir(), ?, stage="fetch") at main.py:789 and
main.py:1402 ? leaked copies shrink exactly the headroom the fetch guard checks, so accumulation
converts into false fetch rejections. Windows does not auto-reclaim %TEMP% by default, so the
docstring premise is false on the shipping platform. Severity P2 honest: affects only remote-

```

input flows (local paths are identity passthrough, __init__.py:85-88), slow-burn, no data loss.
Not owner-gated ? pure disk hygiene, no strategy/spend/ToS/corpus decision.
204
205     ### R1-18 [P2/S] critical-fix ? badminton-highlight-indexer
206     **GDriveApiStorage.fetch_to_local buffers the entire file in RAM
(get_media().execute()); put() deletes the old object before the new upload succeeds**
207     - anchor: `backend/storage/gdrive_api.py:144` · tracked in: ? · finder:
engine:correctness
208     - evidence: gdrive_api.py:143-148: `data =
self._service.files().get_media(fileId=file_id).execute()` then `fh.write(data)` ? the whole
object is materialized as one bytes value (no MediaIoBaseDownload streaming).
gdrive_api.py:163-170: put() resolves an existing file at the path and `files().delete(...)`
BEFORE `files().create(...)` ? delete-then-create is not atomic.
209     - why it matters: Latent OOM + data-loss on the Drive-API storage hot path before it
goes live for the reel round-trip.
210     - proposed fix: Stream downloads with googleapiclient.http.MediaIoBaseDownload in
chunks; for put(), create the new file first (or upload to a temp name) and delete the old one
only after success.
211     - verifier notes: File/line accurate (line 144 is the get_media().execute() call).
Failure scenarios: (1) fetch_to_local on a multi-GB source/reel buffers the full file in RAM on
a memory-limited Cloud Run box ? OOM kill; (2) re-put() of a reel to an existing path deletes
the old file, then the resumable create fails (network/quota) ? the reel is gone with no copy at
either end. Fix refinement: for put(), prefer `files().update(fileId=existing, media_body=...)`
when a file already exists ? atomic in-place content replacement that preserves the file id and
avoids both the delete-first data-loss window and the duplicate-(name,parent) window that
create-first-then-delete would open (the file's own comment at lines 160-162 notes _resolve_id
returns the first match among duplicates). Download fix as proposed: MediaIoBaseDownload chunked
streaming.
212     - verifier reasoning: backend/storage/gdrive_api.py:143-148 ? fetch_to_local does `data
= self._service.files().get_media(fileId=file_id).execute()` then `fh.write(data)`: the whole
object is returned as a single bytes value; no MediaIoBaseDownload/chunked loop exists in the
file (only MediaFileUpload for uploads, lines 78-84). Docstring (lines 3-6, 19) states this
backend serves videos/reels on headless Cloud Run/GPU hosts, so multi-GB buffering is a genuine
OOM risk. backend/storage/gdrive_api.py:163-170 ? put() resolves `existing`, calls
`files().delete(fileId=existing).execute()`, and only then `files().create(...)` : a create
failure after the delete loses the prior object (non-atomic overwrite). git log --
backend/storage/gdrive_api.py shows only formatting/isort commits (b6214e5, f523d3a) since the
introducing commit 819e395 (M12, PR #338) ? not fixed. Not owner-gated: the module's owner gate
(lines 14-17) covers the per-user OAuth flow, not this code's download/upload mechanics.
213
214     ### R1-19 [P2/S] critical-fix ? badminton-highlight-indexer

```

6. user (2026-07-02T04:54:42.502Z)

```

diff --git a/backend/storage/gdrive_api.py b/backend/storage/gdrive_api.py
index 210edce..3252e39 100644
--- a/backend/storage/gdrive_api.py
+++ b/backend/storage/gdrive_api.py
@@ -18,9 +18,12 @@

```

```

    **Scope caveats (M12):** handles **binary** files (videos / reels / JSON) ? Google-native Docs
(Sheets/Slides) have no direct download and would need ``files().export_media`` (out of scope;
export
-them first). ``put`` is idempotent (deletes an existing file at the same path before creating).
Path
-resolution keys on ``(name, parent)``; if Drive already holds duplicate-named files under one
folder
-(a rare pre-existing state this backend never creates), resolution returns the first match.
+them first). ``put`` is idempotent: an existing file at the same path is overwritten **in
place** via
+``files().update`` (same file id; the old bytes survive until the new upload succeeds ? never
+delete-then-create). Path resolution keys on ``(name, parent)``; if Drive already holds
+duplicate-named files under one folder (a rare pre-existing state this backend never creates),
+resolution returns the first match. Downloads stream in chunks (``MediaIoBaseDownload``) ? a
+multi-GB reel is never buffered whole in RAM.

```

```
"""
```

```
from __future__ import annotations
```

```
@@ -35,6 +38,9 @@
```

```
_FOLDER_MIME = "application/vnd.google-apps.folder"
```

```
_DRIVE_SCOPE = "https://www.googleapis.com/auth/drive"
```

```
## Download chunk size: bounds fetch_to_local RAM no matter the object size (vs. the  
googleapiclient
```

```
## default of 100 MiB) while keeping the number of round-trips per multi-GB reel reasonable.
```

```
+_DOWNLOAD_CHUNK_BYTES = 32 * 1024 * 1024
```

```
def _q_escape(name: str) -> str:
```

```
@@ -83,6 +89,18 @@ def _media_body(self, local_path: str, content_type: Optional[str]) -> Any:  
    local_path, mimetype=content_type, resumable=True  
    ) # pragma: no cover
```

```
+ def _downloader(self, fh: Any, request: Any) -> Any:
```

```
+ """The chunked-download driver for a ``files().get_media`` request ? lazy  
+ ``MediaIoBaseDownload`` (real path). A test seam: monkeypatch this to avoid the  
+ ``googleapiclient`` dependency (the fakes drive the same ``next_chunk()`` loop)."""  
+ from googleapiclient.http import (  
+     MediaIoBaseDownload, # type: ignore # pragma: no cover  
+ )
```

```
+ return MediaIoBaseDownload(  
+     fh, request, chunksize=_DOWNLOAD_CHUNK_BYTES  
+ ) # pragma: no cover  
+
```

```
# -- path -> id resolution
```

```
-----  
def _resolve_id(self, key: str, *, create_dirs: bool = False) -> Optional[str]:  
    """Walk ``key``'s path components from the root folder, returning the leaf's file id  
(or
```

```
@@ -140,12 +158,24 @@ def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
```

```
    file_id = self._resolve_id(ref.key)  
    if not file_id:  
        raise FileNotFoundError(f"gdrive-api: {ref.uri!r} not found")  
- data = (  
-     self._service.files().get_media(fileId=file_id).execute()  
- ) # alt=media ? raw bytes  
+ request = self._service.files().get_media(fileId=file_id) # alt=media ? bytes  
    os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)  
- with io.open(dst, "wb") as fh:  
-     fh.write(data)  
+ # Stream in bounded chunks to a ``.part`` sibling, then rename into place: a multi-GB  
+ # reel never sits whole in RAM (the old ``get_media().execute()`` buffered the entire  
+ # object as one bytes value ? OOM fodder on a memory-limited Cloud Run box), and a  
+ # mid-download failure can't leave a truncated ``dst`` that a later ``overwrite=False``  
+ # call would trust as complete.  
+ tmp = dst + ".part"  
+ try:  
+     with io.open(tmp, "wb") as fh:  
+         downloader = self._downloader(fh, request)  
+         done = False  
+         while not done:  
+             _, done = downloader.next_chunk()  
+         os.replace(tmp, dst)  
+ finally:  
+     if os.path.exists(tmp):  
+         os.remove(tmp)  
    return dst
```

```
def put(self, local_path, ref, *, content_type=None) -> StorageRef:
```

```

@@ -158,16 +188,22 @@ def put(self, local_path, ref, *, content_type=None) -> StorageRef:
    if parent_id is None:
        raise FileNotFoundError(f"gdrive-api: cannot resolve parent of {ref.uri!r}")
    # Idempotent: Drive allows duplicate (name, parent), so a plain create() on a re-put
would
-    # ORPHAN the prior file (and reads would keep returning the stale first match). Delete
an
-    # existing file at this exact path first, so put() overwrites rather than accumulates.
+    # ORPHAN the prior file (and reads would keep returning the stale first match).
Overwrite
+    # an existing file at this exact path IN PLACE via files().update ? the file id is kept
+    # and the OLD bytes survive until the NEW upload succeeds. (The previous delete-then-
create
+    # ordering lost the object outright when the create failed mid-upload; create-first-
then-
+    # delete would instead open a duplicate-(name, parent) window where reads could resolve
+    # either copy.)
+    media = self._media_body(str(local_path), content_type)
    existing = self._resolve_id(ref.key)
    if existing:
-        self._service.files().delete(fileId=existing).execute()
-    self._service.files().create(
-        body={"name": name, "parents": [parent_id]},
-        media_body=self._media_body(str(local_path), content_type),
-        fields="id",
-    ).execute()
+    self._service.files().update(fileId=existing, media_body=media).execute()
+    else:
+        self._service.files().create(
+            body={"name": name, "parents": [parent_id]},
+            media_body=media,
+            fields="id",
+        ).execute()
    return ref

```

```

def exists(self, ref) -> bool:
diff --git a/tests/test_storage_gdrive_api.py b/tests/test_storage_gdrive_api.py
index c89af22..44b001f 100644
--- a/tests/test_storage_gdrive_api.py
+++ b/tests/test_storage_gdrive_api.py
@@ -2,8 +2,10 @@

```

Fully offline: a tiny in-memory ``\_FakeDrive`` stands in for the googleapiclient Drive v3 ``service`` (injected, mirroring the GCS fake-client tests). No ``google-api-python-client`` / network needed.

-Asserts: scheme parsing, registry wiring, the hyphen?underscore settings lookup, and the full -path-keyed round-trip (put ? exists ? stat ? fetch ? list ? delete).

+Asserts: scheme parsing, registry wiring, the hyphen?underscore settings lookup, the full +path-keyed round-trip (put ? exists ? stat ? fetch ? list ? delete), the chunked (streaming) +download path, and overwrite safety: a re-put updates in place (same file id) and a mid-upload +failure leaves the OLD object intact ? never delete-before-upload.

"""

```

import re
@@ -26,6 +28,17 @@ def execute(self):
    return self._value

```

```

+class _RaiseExec:
+    """An operation whose ``execute()`` fails (simulated network/quota mid-upload error) ?
+    crucially WITHOUT having mutated the fake Drive first."""
+
+    def __init__(self, msg):
+        self._msg = msg

```



```

+
+     def execute(self):
+         raise RuntimeError(self._msg)
+
+
+ class _FakeFiles:
+     def __init__(self, drive):
+         self.d = drive
@@ -46,6 +59,8 @@ def list(self, q="", spaces=None, fields=None, pageToken=None):
+         return _Exec({"files": out}) # one page, no nextPageToken

+     def create(self, body=None, media_body=None, fields=None):
+         if media_body is not None and self.d.fail_uploads:
+             return _RaiseExec("simulated mid-upload failure (network/quota)")
+         nid = self.d.new_id()
+         self.d.nodes[nid] = {
+             "name": body["name"],
@@ -59,6 +74,13 @@ def create(self, body=None, media_body=None, fields=None):
+             self.d.content[nid] = fh.read()
+         return _Exec({"id": nid})

+     def update(self, fileId=None, media_body=None, fields=None):
+         if self.d.fail_uploads:
+             return _RaiseExec("simulated mid-upload failure (network/quota)")
+         with open(media_body, "rb") as fh: # _media_body patched ? the local path
+             self.d.content[fileId] = fh.read()
+         return _Exec({"id": fileId})
+
+     def get_media(self, fileId=None):
+         return _Exec(self.d.content.get(fileId, b""))

@@ -85,6 +107,9 @@ def __init__(self):
+         self.nodes = {"root": {"name": "root", "mimeType": "folder", "parents": []}}
+         self.content = {}
+         self._n = 0
+         self.fail_uploads = (
+             False # flip on to make the NEXT upload (create/update) fail
+         )

+     def new_id(self):
+         self._n += 1
@@ -94,11 +119,36 @@ def files(self):
+         return _FakeFiles(self)

+
+class _FakeDownloader:
+    """Stands in for ``googleapiclient.http.MediaIoBaseDownload``: drains the fake
+    ``get_media``
+    request's bytes into the file handle a few bytes per ``next_chunk()`` call, so the
+    production streaming loop is exercised chunk by chunk (never one whole-file buffer)."""
+
+    chunk_size = 4
+
+    def __init__(self, fh, request):
+        self._fh = fh
+        self._data = request.execute() # the fake request; a real one streams over HTTP
+        self._pos = 0
+        self.chunks = 0
+
+    def next_chunk(self):
+        piece = self._data[self._pos : self._pos + self.chunk_size]
+        self._fh.write(piece)
+        self._pos += len(piece)
+        self.chunks += 1
+        return None, self._pos >= len(self._data)

```

```

+
+
@pytest.fixture
def be(monkeypatch):
-     """A GDriveApiStorage on a fake service, with _media_body patched to the local path (no
-     googleapiclient dependency ? the fake `create` reads the bytes from that path)."""
+     """A GDriveApiStorage on a fake service, with the two googleapiclient seams patched:
+     _media_body ? the local path (the fake `create`/`update` read the bytes from it) and
+     _downloader ? _FakeDownloader (drives the real next_chunk() streaming loop)."""
    monkeypatch.setattr(GDriveApiStorage, "_media_body", lambda self, p, ct: p)
+    monkeypatch.setattr(
+        GDriveApiStorage, "_downloader", lambda self, fh, req: _FakeDownloader(fh, req)
+    )
    return GDriveApiStorage(service=_FakeDrive())

@@ -163,7 +213,7 @@ def test_roundtrip_put_exists_stat_fetch_list_delete(be, tmp_path):

def test_put_is_idempotent_no_duplicate_on_reput(be, tmp_path):
-     """Re-putting the same path OVERWRITES (delete-then-create) ? Drive allows duplicate names,
so a
+     """Re-putting the same path OVERWRITES (update-in-place) ? Drive allows duplicate names, so
a
    naive create would orphan the old file and reads would keep returning the stale one."""
    ref = parse_uri("gdrive-api://Coaching/clip.mp4")
    v1 = tmp_path / "v1.mp4"
@@ -181,6 +231,98 @@ def test_put_is_idempotent_no_duplicate_on_reput(be, tmp_path):
    ) # the latest bytes, not the orphaned first

+def test_reput_updates_in_place_preserving_file_id(be, tmp_path):
+     """The idempotent overwrite is files().update on the EXISTING id ? the Drive file id is
+     stable across re-puts (shared links / ids held elsewhere keep working)."""
+     ref = parse_uri("gdrive-api://Coaching/clip.mp4")
+     v1 = tmp_path / "v1.mp4"
+     v1.write_bytes(b"FIRST")
+     v2 = tmp_path / "v2.mp4"
+     v2.write_bytes(b"SECOND")
+     be.put(str(v1), ref)
+     id_before = be._resolve_id(ref.key)
+     be.put(str(v2), ref)
+     assert be._resolve_id(ref.key) == id_before
+
+
+def test_put_failure_mid_upload_keeps_old_object_intact(be, tmp_path):
+     """A re-put whose upload fails must leave the PREVIOUS object readable. The old
+     delete-then-create ordering deleted first, so a failed create lost the reel at both ends;
+     update-in-place only replaces content once the upload succeeds."""
+     ref = parse_uri("gdrive-api://Coaching/reel.mp4")
+     v1 = tmp_path / "v1.mp4"
+     v1.write_bytes(b"OLD-REEL")
+     v2 = tmp_path / "v2.mp4"
+     v2.write_bytes(b"NEW-REEL")
+     be.put(str(v1), ref)
+
+     be._service.fail_uploads = True
+     with pytest.raises(RuntimeError, match="mid-upload"):
+         be.put(str(v2), ref)
+
+     assert be.exists(ref) # still there ?
+     dst = tmp_path / "out.mp4"
+     be.fetch_to_local(ref, str(dst))
+     assert dst.read_bytes() == b"OLD-REEL" # ? with the old bytes intact
+

```

```

+     be._service.fail_uploads = False # and a retry succeeds: still exactly one file
+     be.put(str(v2), ref)
+     listed = [r.uri for r in be.list(parse_uri("gdrive-api://Coaching"))]
+     assert listed == ["gdrive-api://Coaching/reel.mp4"]
+     dst2 = tmp_path / "out2.mp4"
+     be.fetch_to_local(ref, str(dst2))
+     assert dst2.read_bytes() == b"NEW-REEL"
+
+
+def test_fetch_large_file_streams_in_chunks(be, tmp_path, monkeypatch):
+    """A file much larger than the chunk size is drained via MANY next_chunk() calls ? the
+    production loop streams; it never materializes the object as one .execute() buffer."""
+    data = b"0123456789" * 5 # 50 bytes ? the fake 4-byte chunk
+    src = tmp_path / "big.mp4"
+    src.write_bytes(data)
+    ref = parse_uri("gdrive-api://Coaching/big.mp4")
+    be.put(str(src), ref)
+
+    made = []
+
+    def spy(self, fh, req):
+        d = _FakeDownloader(fh, req)
+        made.append(d)
+        return d
+
+    monkeypatch.setattr(GDriveApiStorage, "_downloader", spy)
+    dst = tmp_path / "out" / "big.mp4"
+    be.fetch_to_local(ref, str(dst))
+    assert dst.read_bytes() == data # byte-identical after reassembly
+    assert made[0].chunks == 13 # ceil(50 / 4) ? chunked, not one whole-file buffer
+    assert not (tmp_path / "out" / "big.mp4.part").exists() # temp renamed away
+
+
+def test_fetch_failure_mid_download_leaves_no_truncated_dst(be, tmp_path, monkeypatch):
+    """If the stream dies mid-download, dst must NOT exist (a truncated file would be trusted
+    as complete by a later overwrite=False call) and the .part temp is cleaned up."""
+
+    class _Boom(_FakeDownloader):
+        def next_chunk(self):
+            if self.chunks >= 2:
+                raise RuntimeError("simulated mid-download network failure")
+            return super().next_chunk()
+
+    src = tmp_path / "big.mp4"
+    src.write_bytes(b"0123456789ABCDEF")
+    ref = parse_uri("gdrive-api://Coaching/big.mp4")
+    be.put(str(src), ref)
+
+    monkeypatch.setattr(
+        GDriveApiStorage, "_downloader", lambda self, fh, req: _Boom(fh, req)
+    )
+    dst = tmp_path / "out.mp4"
+    with pytest.raises(RuntimeError, match="mid-download"):
+        be.fetch_to_local(ref, str(dst))
+    assert not dst.exists()
+    assert not (tmp_path / "out.mp4.part").exists()
+
+
+def test_fetch_missing_raises(be, tmp_path):
+    with pytest.raises(FileNotFoundError):
+        be.fetch_to_local(

```

7. user (2026-07-02T04:54:43.493Z)

backend/storage/gdrive_api.py

tests/test_storage_gdrive_api.py

8. assistant (2026-07-02T04:54:49.680Z)

Now let me read the full source file in the worktree for context.

9. user (2026-07-02T04:54:52.088Z)

```
1      """Google Drive **API** backend (OAuth / service-account) ? ``gdrive-api://`` (M12, D14
2      Northstar).
3      The cloud-native Drive round-trip: read a source from + write the stitched reel **back
4      to** a user's
5      Google Drive **via the Drive API** ? no local mount required (unlike the ``gdrive://``
6      backend, which
7      is plain local-FS over a *mounted* Drive for Desktop and stays untouched). This is the
8      headless path a
9      Cloud Run / GPU box uses.
10     ``gdrive-api://<path>`` names a path **relative to a root folder** (default the Drive
11     ``root``, or
12     ``storage.gdrive-api.root_folder_id``); the backend resolves each path component to a
13     Drive file id via
14     ``files().list``. The Google libraries (``google-api-python-client`` + ``google-auth``)
15     are imported
16     **lazily** ? ``import backend.storage`` never pulls them ? and a ``service`` may be
17     **injected** for
18     fully-offline tests (mirroring ``GCSStorage(client=...)``).
19     ? **Credentials are never in config.** The real service is built from Application
20     Default Credentials
21     (a service account / workload identity) over the ``drive`` scope. **Per-user interactive
22     OAuth (the
23     consumer "connect your Drive" flow: client id/secret + refresh tokens) is OWNER-gated**
24     ? this backend
25     ships the service-account/ADC build + the injected fake; the live per-user OAuth app is
26     a follow-up.
27     **Scope caveats (M12):** handles **binary** files (videos / reels / JSON) ? Google-
28     native Docs
29     (Sheets/Slides) have no direct download and would need ``files().export_media`` (out of
30     scope; export
31     them first). ``put`` is idempotent: an existing file at the same path is overwritten
32     **in place** via
33     ``files().update`` (same file id; the old bytes survive until the new upload succeeds ?
34     never
35     delete-then-create). Path resolution keys on ``(name, parent)``; if Drive already holds
36     duplicate-named files under one folder (a rare pre-existing state this backend never
37     creates),
38     resolution returns the first match. Downloads stream in chunks (``MediaIoBaseDownload``)
39     ? a
40     multi-GB reel is never buffered whole in RAM.
41     """
42
43     from __future__ import annotations
44
45     import io
46     import os
47     from datetime import datetime
48     from typing import Any, Iterator, Optional
49
50     from backend.storage.base import StorageBackend
51     from backend.storage.ref import StorageRef, StorageStat
52
53     _FOLDER_MIME = "application/vnd.google-apps.folder"
```

```

40     _DRIVE_SCOPE = "https://www.googleapis.com/auth/drive"
41     # Download chunk size: bounds fetch_to_local RAM no matter the object size (vs. the
googleapiclient
42     # default of 100 MiB) while keeping the number of round-trips per multi-GB reel
reasonable.
43     _DOWNLOAD_CHUNK_BYTES = 32 * 1024 * 1024
44
45
46     def _q_escape(name: str) -> str:
47         """Escape a value for a Drive ``q`` string literal (single-quoted): ``\`` then
``\``,\"""
48         return name.replace("\\", "\\").replace("'", "\\'")
49
50
51     class GDriveApiStorage(StorageBackend):
52         """Drive-API-backed storage. Construct with an injected ``service`` (tests) or let
it build one
53         from ADC (production). ``root_folder_id`` anchors relative ``gdrive-api://``
paths."""
54
55         def __init__(
56             self, *, service: Any = None, root_folder_id: str = "root", **_ignored
57         ) -> None:
58             self._root_id = root_folder_id or "root"
59             if service is not None:
60                 self._service = service
61                 return
62             self._service = self._build_service()
63
64         # -- construction (real path; lazy Google imports)
-----
65         def _build_service(self) -> Any:
66             try:
67                 import google.auth # type: ignore
68                 from googleapiclient.discovery import build # type: ignore
69             except (
70                 ImportError
71             ) as e: # pragma: no cover - exercised only without the extra installed
72                 raise RuntimeError(
73                     "the gdrive-api backend needs the 'storage-gdrive-api' extra "
74                     "(google-api-python-client + google-auth). Install it on a host that
uses "
75                     "gdrive-api:// URIs; credentials come from ADC (a service account /
workload "
76                     "identity over the Drive scope) ? never from config."
77                 ) from e
78             creds, _ = google.auth.default(scopes=[_DRIVE_SCOPE])
79             return build("drive", "v3", credentials=creds, cache_discovery=False)
80
81         def _media_body(self, local_path: str, content_type: Optional[str]) -> Any:
82             """The upload media object for ``files().create`` ? lazy ``MediaFileUpload``
(real path).
83             A test seam: monkeypatch this to avoid the ``googleapiclient`` dependency."""
84             from googleapiclient.http import (
85                 MediaFileUpload, # type: ignore # pragma: no cover
86             )
87
88             return MediaFileUpload(
89                 local_path, mimetype=content_type, resumable=True
90             ) # pragma: no cover
91
92         def _downloader(self, fh: Any, request: Any) -> Any:
93             """The chunked-download driver for a ``files().get_media`` request ? lazy
``MediaIoBaseDownload`` (real path). A test seam: monkeypatch this to avoid the
94             ``googleapiclient`` dependency (the fakes drive the same ``next_chunk()``
95

```

```

loop)."""
96         from googleapiclient.http import (
97             MediaIoBaseDownload, # type: ignore # pragma: no cover
98         )
99
100        return MediaIoBaseDownload(
101            fh, request, chunksize=_DOWNLOAD_CHUNK_BYTES
102        ) # pragma: no cover
103
104        # -- path -> id resolution
105        -----
106        def _resolve_id(self, key: str, *, create_dirs: bool = False) -> Optional[str]:
107            """Walk ``key``'s path components from the root folder, returning the leaf's
108            file id (or
109            ``None`` if a component is missing). With ``create_dirs`` EVERY missing
110            component is created as
111            a folder ? ``put`` uses this on the parent-directory path (where all components
112            are folders),
113            so a write into a fresh subfolder works. The read paths leave it ``False`` (a
114            missing component
115            ? ``None``), never creating anything."""
116            parent = self._root_id
117            parts = [p for p in key.split("/") if p]
118            for part in parts:
119                q = (
120                    f"name = '{_q_escape(part)}' and '{_q_escape(parent)}' in parents "
121                    f"and trashed = false"
122                )
123                found = (
124                    self._service.files()
125                    .list(q=q, spaces="drive", fields="files(id, mimeType)")
126                    .execute()
127                    .get("files", [])
128                )
129                if found:
130                    parent = found[0]["id"]
131                elif create_dirs:
132                    parent = (
133                        self._service.files()
134                        .create(
135                            body={
136                                "name": part,
137                                "mimeType": _FOLDER_MIME,
138                                "parents": [parent],
139                            },
140                            fields="id",
141                        )
142                        .execute()["id"]
143                    )
144                else:
145                    return None
146            return parent
147
148        def _split(self, key: str) -> tuple[str, str]:
149            """Split a ``gdrive-api://`` key into (parent dir path, leaf name)."""
150            clean = key.strip("/")
151            head, _, name = clean.rpartition("/")
152            return head, name
153
154        # -- StorageBackend interface
155        -----
156        def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
157            if dst is None:
158                raise ValueError(
159                    "GDriveApiStorage.fetch_to_local requires a dst path (no local mount)"
160                )

```

```

154         )
155         dst = str(dst)
156         if os.path.exists(dst) and not overwrite:
157             return dst
158         file_id = self._resolve_id(ref.key)
159         if not file_id:
160             raise FileNotFoundError(f"gdrive-api: {ref.uri!r} not found")
161         request = self._service.files().get_media(fileId=file_id) # alt=media ? bytes
162         os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
163         # Stream in bounded chunks to a ``.part`` sibling, then rename into place: a
multi-GB
164         # reel never sits whole in RAM (the old ``get_media().execute()`` buffered the
entire
165         # object as one bytes value ? OOM fodder on a memory-limited Cloud Run box), and
a
166         # mid-download failure can't leave a truncated ``dst`` that a later
``overwrite=False``
167         # call would trust as complete.
168         tmp = dst + ".part"
169         try:
170             with io.open(tmp, "wb") as fh:
171                 downloader = self._downloader(fh, request)
172                 done = False
173                 while not done:
174                     _, done = downloader.next_chunk()
175                 os.replace(tmp, dst)
176         finally:
177             if os.path.exists(tmp):
178                 os.remove(tmp)
179         return dst
180
181     def put(self, local_path, ref, *, content_type=None) -> StorageRef:
182         parent_path, name = self._split(ref.key)
183         parent_id = (
184             self._resolve_id(parent_path, create_dirs=True)
185             if parent_path
186             else self._root_id
187         )
188         if parent_id is None:
189             raise FileNotFoundError(f"gdrive-api: cannot resolve parent of {ref.uri!r}")
190         # Idempotent: Drive allows duplicate (name, parent), so a plain create() on a
re-put would
191         # ORPHAN the prior file (and reads would keep returning the stale first match).
Overwrite
192         # an existing file at this exact path IN PLACE via files().update ? the file id
is kept
193         # and the OLD bytes survive until the NEW upload succeeds. (The previous delete-
then-create
194         # ordering lost the object outright when the create failed mid-upload; create-
first-then-
195         # delete would instead open a duplicate-(name, parent) window where reads could
resolve
196         # either copy.)
197         media = self._media_body(str(local_path), content_type)
198         existing = self._resolve_id(ref.key)
199         if existing:
200             self._service.files().update(fileId=existing, media_body=media).execute()
201         else:
202             self._service.files().create(
203                 body={"name": name, "parents": [parent_id]},
204                 media_body=media,
205                 fields="id",
206             ).execute()
207         return ref
208

```

```

209     def exists(self, ref) -> bool:
210         return self._resolve_id(ref.key) is not None
211
212     def stat(self, ref) -> StorageStat:
213         file_id = self._resolve_id(ref.key)
214         if not file_id:
215             raise FileNotFoundError(f"gdrive-api: {ref.uri!r} not found")
216         meta = (
217             self._service.files()
218             .get(fileId=file_id, fields="size, modifiedTime, md5Checksum, mimeType")
219             .execute()
220         )
221         modified = None
222         if meta.get("modifiedTime"):
223             # Drive returns RFC-3339 (e.g. "2026-06-22T10:00:00.000Z"); normalize the
trailing Z.
224             modified = datetime.fromisoformat(
225                 meta["modifiedTime"].replace("Z", "+00:00")
226             )
227         return StorageStat(
228             size=int(meta.get("size") or 0),
229             modified=modified,
230             etag=meta.get("md5Checksum"),
231             content_type=meta.get("mimeType"),
232         )
233
234     def list(self, prefix) -> Iterator[StorageRef]:
235         folder_id = (
236             self._resolve_id(prefix.key) if prefix.key.strip("/") else self._root_id
237         )
238         if not folder_id:
239             return
240         base = prefix.key.strip("/")
241         page_token: Optional[str] = None
242         while True:
243             resp = (
244                 self._service.files()
245                 .list(
246                     q=f'"{_q_escape(folder_id)}" in parents and trashed = false',
247                     spaces="drive",
248                     fields="nextPageToken, files(id, name)",
249                     pageToken=page_token,
250                 )
251                 .execute()
252             )
253             for f in resp.get("files", []):
254                 key = f"{base}/{f['name']}" if base else f["name"]
255                 yield StorageRef("gdrive-api", "", key, f"gdrive-api://{key}")
256             page_token = resp.get("nextPageToken")
257             if not page_token:
258                 break
259
260     def delete(self, ref) -> None:
261         file_id = self._resolve_id(ref.key)
262         if file_id:
263             self._service.files().delete(fileId=file_id).execute()
264

```

10. user (2026-07-02T04:56:13.127Z)

6afacdd fix(storage): gdrive-api ? chunked streaming download + loss-proof idempotent put
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
fix/gdrive-api-chunked-download-safe-put

11. user (2026-07-02T04:56:15.005Z)

```
{"baseRefName": "master", "headRefName": "fix/gdrive-api-chunked-download-safe-put", "headRefOid": "6afacdd8e3974200391da65da16cdcda3ee7df39"}
```

12. user (2026-07-02T04:56:25.232Z)

```
backend/storage/gdrive_api.py      | 68 ++++++++-----
tests/test_storage_gdrive_api.py   | 152 ++++++++-----
2 files changed, 199 insertions(+), 21 deletions(-)
6afacdd fix(storage): gdrive-api ? chunked streaming download + loss-proof idempotent put
```

13. user (2026-07-02T04:56:26.858Z)

```
..... [100%]
14 passed in 0.43s
```

14. user (2026-07-02T04:56:40.006Z)

```
..... [100%]
57 passed, 2 skipped, 1505 deselected in 4.91s
```

15. user (2026-07-02T04:56:41.080Z)

```
All checks passed!
2 files already formatted
Success: no issues found in 1 source file
```

16. user (2026-07-02T04:56:53.183Z)

```
Command running in background with ID: b363oyqi5. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\tasks\b363oyqi5.output. You will be notified when it completes. To check interim output, use Read on that file path.
```

17. user (2026-07-02T04:56:54.651Z)

```
1      """M12 ? the ``gdrive-api://`` OAuth/API StorageBackend (D14 Northstar).
2
3      Fully offline: a tiny in-memory ``_FakeDrive`` stands in for the googleapiclient Drive
v3 ``service``
4      (injected, mirroring the GCS fake-client tests). No ``google-api-python-client`` /
network needed.
5      Asserts: scheme parsing, registry wiring, the hyphen?underscore settings lookup, the
full
6      path-keyed round-trip (put ? exists ? stat ? fetch ? list ? delete), the chunked
(streaming)
7      download path, and overwrite safety: a re-put updates in place (same file id) and a mid-
upload
8      failure leaves the OLD object intact ? never delete-before-upload.
9      """
10
11     import re
12
13     import pytest
14
15     from backend.config import AppConfig
16     from backend.storage import get_storage
17     from backend.storage.gdrive_api import GDriveApiStorage
18     from backend.storage.ref import parse_uri
19
20     # ----- the in-memory fake
Drive service
21
```

```

22
23 class _Exec:
24     def __init__(self, value):
25         self._value = value
26
27     def execute(self):
28         return self._value
29
30
31 class _RaiseExec:
32     """An operation whose ``execute()`` fails (simulated network/quota mid-upload error)
33     ?
34     crucially WITHOUT having mutated the fake Drive first."""
35
36     def __init__(self, msg):
37         self._msg = msg
38
39     def execute(self):
40         raise RuntimeError(self._msg)
41
42 class _FakeFiles:
43     def __init__(self, drive):
44         self.d = drive
45
46     def list(self, q="", spaces=None, fields=None, pageToken=None):
47         name_m = re.search(r"name = '([^\']*)'", q)
48         parent_m = re.search(r"'([^\']*)' in parents", q)
49         parent = parent_m.group(1) if parent_m else None
50         out = []
51         for nid, n in self.d.nodes.items():
52             if parent is not None and parent not in n["parents"]:
53                 continue
54             if name_m and n["name"] != name_m.group(1):
55                 continue
56             if nid == "root":
57                 continue
58             out.append({"id": nid, "name": n["name"], "mimeType": n["mimeType"]})
59         return _Exec({"files": out}) # one page, no nextPageToken
60
61     def create(self, body=None, media_body=None, fields=None):
62         if media_body is not None and self.d.fail_uploads:
63             return _RaiseExec("simulated mid-upload failure (network/quota)")
64         nid = self.d.new_id()
65         self.d.nodes[nid] = {
66             "name": body["name"],
67             "mimeType": body.get("mimeType", "application/octet-stream"),
68             "parents": list(body.get("parents", [])),
69         }
70         if (
71             media_body is not None
72         ): # in tests _media_body is patched to return the local path
73             with open(media_body, "rb") as fh:
74                 self.d.content[nid] = fh.read()
75         return _Exec({"id": nid})
76
77     def update(self, fileId=None, media_body=None, fields=None):
78         if self.d.fail_uploads:
79             return _RaiseExec("simulated mid-upload failure (network/quota)")
80         with open(media_body, "rb") as fh: # _media_body patched ? the local path
81             self.d.content[fileId] = fh.read()
82         return _Exec({"id": fileId})
83
84     def get_media(self, fileId=None):
85         return _Exec(self.d.content.get(fileId, b""))

```

```

86
87     def get(self, fileId=None, fields=None):
88         n = self.d.nodes[fileId]
89         data = self.d.content.get(fileId, b"")
90         return _Exec(
91             {
92                 "size": str(len(data)),
93                 "modifiedTime": "2026-06-22T10:00:00.000Z",
94                 "md5Checksum": "deadbeef",
95                 "mimeType": n["mimeType"],
96             }
97         )
98
99     def delete(self, fileId=None):
100         self.d.nodes.pop(fileId, None)
101         self.d.content.pop(fileId, None)
102         return _Exec({})
103
104
105 class _FakeDrive:
106     def __init__(self):
107         self.nodes = {"root": {"name": "root", "mimeType": "folder", "parents": []}}
108         self.content = {}
109         self._n = 0
110         self.fail_uploads = (
111             False # flip on to make the NEXT upload (create/update) fail
112         )
113
114     def new_id(self):
115         self._n += 1
116         return f"id{self._n}"
117
118     def files(self):
119         return _FakeFiles(self)
120
121
122 class _FakeDownloader:
123     """Stands in for ``googleapiclient.http.MediaIoBaseDownload``: drains the fake
124     ``get_media``
125     request's bytes into the file handle a few bytes per ``next_chunk()`` call, so the
126     production streaming loop is exercised chunk by chunk (never one whole-file
127     buffer)."""
128
129     chunk_size = 4
130
131     def __init__(self, fh, request):
132         self.fh = fh
133         self.data = request.execute() # the fake request; a real one streams over HTTP
134         self._pos = 0
135         self.chunks = 0
136
137     def next_chunk(self):
138         piece = self._data[self._pos : self._pos + self.chunk_size]
139         self.fh.write(piece)
140         self._pos += len(piece)
141         self.chunks += 1
142         return None, self._pos >= len(self._data)
143
144 @pytest.fixture
145 def be(monkeypatch):
146     """A GDriveApiStorage on a fake service, with the two googleapiclient seams patched:
147     _media_body ? the local path (the fake `create`/`update` read the bytes from it) and
148     _downloader ? _FakeDownloader (drives the real next_chunk() streaming loop)."""
149     monkeypatch.setattr(GDriveApiStorage, "_media_body", lambda self, p, ct: p)

```

```

149     monkeypatch.setattr(
150         GDriveApiStorage, "_downloader", lambda self, fh, req: _FakeDownloader(fh, req)
151     )
152     return GDriveApiStorage(service=_FakeDrive())
153
154
155     # ----- scheme +
registry wiring
156
157
158     def test_parse_uri_gdrive_api_is_path_keyed():
159         ref = parse_uri("gdrive-api://Coaching/2026/clip.mp4")
160         assert (ref.backend, ref.bucket, ref.key) == (
161             "gdrive-api",
162             "",
163             "Coaching/2026/clip.mp4",
164         )
165         assert ref.uri == "gdrive-api://Coaching/2026/clip.mp4"
166
167
168     def test_unknown_scheme_still_rejected():
169         with pytest.raises(ValueError):
170             parse_uri("gdrive-zzz://x")
171
172
173     def test_config_accepts_gdrive_api_backend():
174         cfg = AppConfig.model_validate(
175             {"storage": {"backend": "gdrive-api", "input_backend": "gdrive-api"}}
176         )
177         assert cfg.storage.backend == "gdrive-api"
178         assert cfg.storage.input_backend == "gdrive-api"
179
180
181     def test_get_storage_wires_gdrive_api_and_reads_underscore_settings(monkeypatch):
182         """Registry resolves 'gdrive-api' ? GDriveApiStorage, and its settings come from the
183         `gdrive_api` field (hyphen?underscore lookup) merged with overrides (here, the
184         injected service)."""
185         monkeypatch.setattr(GDriveApiStorage, "_media_body", lambda self, p, ct: p)
186         cfg = {"backend": "gdrive-api", "gdrive_api": {"root_folder_id": "FOLDER42"}}
187         be = get_storage("gdrive-api", config=cfg, service=_FakeDrive())
188         assert isinstance(be, GDriveApiStorage)
189         assert be._root_id == "FOLDER42"
190
191     # ----- full
round-trip
192
193
194     def test_roundtrip_put_exists_stat_fetch_list_delete(be, tmp_path):
195         src = tmp_path / "clip.mp4"
196         src.write_bytes(b"VIDEO-BYTES")
197         ref = parse_uri("gdrive-api://Coaching/2026/clip.mp4")
198
199         be.put(str(src), ref) # creates the Coaching/ + 2026/ folders + the file
200         assert be.exists(ref)
201         assert be.stat(ref).size == len(b"VIDEO-BYTES")
202         assert be.stat(ref).etag == "deadbeef"
203
204         dst = tmp_path / "out" / "clip.mp4"
205         be.fetch_to_local(ref, str(dst))
206         assert dst.read_bytes() == b"VIDEO-BYTES"
207
208         listed = [r.uri for r in be.list(parse_uri("gdrive-api://Coaching/2026"))]
209         assert listed == ["gdrive-api://Coaching/2026/clip.mp4"]
210

```

```

211         be.delete(ref)
212         assert not be.exists(ref)
213
214
215     def test_put_is_idempotent_no_duplicate_on_reput(be, tmp_path):
216         """Re-putting the same path OVERWRITES (update-in-place) ? Drive allows duplicate
names, so a
217         naive create would orphan the old file and reads would keep returning the stale
one."""
218         ref = parse_uri("gdrive-api://Coaching/clip.mp4")
219         v1 = tmp_path / "v1.mp4"
220         v1.write_bytes(b"FIRST")
221         v2 = tmp_path / "v2.mp4"
222         v2.write_bytes(b"SECOND-LONGER")
223         be.put(str(v1), ref)
224         be.put(str(v2), ref) # re-put the SAME path
225         listed = [r.uri for r in be.list(parse_uri("gdrive-api://Coaching"))]
226         assert listed == ["gdrive-api://Coaching/clip.mp4"] # exactly ONE file, not two
227         dst = tmp_path / "out.mp4"
228         be.fetch_to_local(ref, str(dst))
229         assert (
230             dst.read_bytes() == b"SECOND-LONGER"
231         ) # the latest bytes, not the orphaned first
232
233
234     def test_reput_updates_in_place_preserving_file_id(be, tmp_path):
235         """The idempotent overwrite is files().update on the EXISTING id ? the Drive file id
is
236         stable across re-puts (shared links / ids held elsewhere keep working)."""
237         ref = parse_uri("gdrive-api://Coaching/clip.mp4")
238         v1 = tmp_path / "v1.mp4"
239         v1.write_bytes(b"FIRST")
240         v2 = tmp_path / "v2.mp4"
241         v2.write_bytes(b"SECOND")
242         be.put(str(v1), ref)
243         id_before = be._resolve_id(ref.key)
244         be.put(str(v2), ref)
245         assert be._resolve_id(ref.key) == id_before
246
247
248     def test_put_failure_mid_upload_keeps_old_object_intact(be, tmp_path):
249         """A re-put whose upload fails must leave the PREVIOUS object readable. The old
250         delete-then-create ordering deleted first, so a failed create lost the reel at both
ends;
251         update-in-place only replaces content once the upload succeeds."""
252         ref = parse_uri("gdrive-api://Coaching/reel.mp4")
253         v1 = tmp_path / "v1.mp4"
254         v1.write_bytes(b"OLD-REEL")
255         v2 = tmp_path / "v2.mp4"
256         v2.write_bytes(b"NEW-REEL")
257         be.put(str(v1), ref)
258
259         be._service.fail_uploads = True
260         with pytest.raises(RuntimeError, match="mid-upload"):
261             be.put(str(v2), ref)
262
263         assert be.exists(ref) # still there ?
264         dst = tmp_path / "out.mp4"
265         be.fetch_to_local(ref, str(dst))
266         assert dst.read_bytes() == b"OLD-REEL" # ? with the old bytes intact
267
268         be._service.fail_uploads = False # and a retry succeeds: still exactly one file
269         be.put(str(v2), ref)
270         listed = [r.uri for r in be.list(parse_uri("gdrive-api://Coaching"))]
271         assert listed == ["gdrive-api://Coaching/reel.mp4"]

```

```

272     dst2 = tmp_path / "out2.mp4"
273     be.fetch_to_local(ref, str(dst2))
274     assert dst2.read_bytes() == b"NEW-REEL"
275
276
277 def test_fetch_large_file_streams_in_chunks(be, tmp_path, monkeypatch):
278     """A file much larger than the chunk size is drained via MANY next_chunk() calls ?
the
279     production loop streams; it never materializes the object as one .execute()
buffer."""
280     data = b"0123456789" * 5 # 50 bytes ? the fake 4-byte chunk
281     src = tmp_path / "big.mp4"
282     src.write_bytes(data)
283     ref = parse_uri("gdrive-api://Coaching/big.mp4")
284     be.put(str(src), ref)
285
286     made = []
287
288     def spy(self, fh, req):
289         d = _FakeDownloader(fh, req)
290         made.append(d)
291         return d
292
293     monkeypatch.setattr(GDriveApiStorage, "_downloader", spy)
294     dst = tmp_path / "out" / "big.mp4"
295     be.fetch_to_local(ref, str(dst))
296     assert dst.read_bytes() == data # byte-identical after reassembly
297     assert made[0].chunks == 13 # ceil(50 / 4) ? chunked, not one whole-file buffer
298     assert not (tmp_path / "out" / "big.mp4.part").exists() # temp renamed away
299
300
301 def test_fetch_failure_mid_download_leaves_no_truncated_dst(be, tmp_path, monkeypatch):
302     """If the stream dies mid-download, dst must NOT exist (a truncated file would be
trusted
303     as complete by a later overwrite=False call) and the .part temp is cleaned up."""
304
305     class _Boom(_FakeDownloader):
306         def next_chunk(self):
307             if self.chunks >= 2:
308                 raise RuntimeError("simulated mid-download network failure")
309             return super().next_chunk()
310
311     src = tmp_path / "big.mp4"
312     src.write_bytes(b"0123456789ABCDEF")
313     ref = parse_uri("gdrive-api://Coaching/big.mp4")
314     be.put(str(src), ref)
315
316     monkeypatch.setattr(
317         GDriveApiStorage, "_downloader", lambda self, fh, req: _Boom(fh, req)
318     )
319     dst = tmp_path / "out.mp4"
320     with pytest.raises(RuntimeError, match="mid-download"):
321         be.fetch_to_local(ref, str(dst))
322     assert not dst.exists()
323     assert not (tmp_path / "out.mp4.part").exists()
324
325
326 def test_fetch_missing_raises(be, tmp_path):
327     with pytest.raises(FileNotFoundError):
328         be.fetch_to_local(
329             parse_uri("gdrive-api://nope/missing.mp4"), str(tmp_path / "x")
330         )
331
332
333 def test_fetch_requires_dst(be):

```

```

334         with pytest.raises(ValueError):
335             be.fetch_to_local(parse_uri("gdrive-api://a/b.mp4"), None)
336
337
338     def test_put_at_root_uses_root_folder(be, tmp_path):
339         src = tmp_path / "top.json"
340         src.write_text("{}")
341         ref = parse_uri("gdrive-api://top.json")
342         be.put(str(src), ref)
343         assert be.exists(ref)
344
345
346     def test_build_service_without_libs_raises_actionable_error(monkeypatch):
347         """No injected service + the Google libs absent ? a clear, owner-actionable error
348         (not ImportError
349         leaking through). googleapiclient is not installed in the test env, so this
350         exercises the real path."""
351         with pytest.raises(RuntimeError, match="storage-gdrive-api"):
352             GDriveApiStorage()

```

18. assistant (2026-07-02T04:57:40.809Z)

While the full suite runs, let me check the repo's coverage config and compare with the sibling GCS backend for convention consistency.

19. user (2026-07-02T04:57:44.677Z)

```

---
backend/storage/gdrive_api.py:71:         ) as e: # pragma: no cover - exercised only without
the extra installed
backend/storage/gdrive_api.py:85:             MediaFileUpload, # type: ignore # pragma: no
cover
backend/storage/gdrive_api.py:90:         ) # pragma: no cover
backend/storage/gdrive_api.py:97:             MediaIoBaseDownload, # type: ignore # pragma: no
cover
backend/storage/gdrive_api.py:102:        ) # pragma: no cover

```

20. user (2026-07-02T04:57:45.789Z)

<persisted-output>

Output too large (35.2KB). Full output saved to: C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\tool-results\toolu_01EyQFB3npUXUTgt5qFaJoeh.txt

Preview (first 2KB):

```

C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-gdrive-api-
safety\backend\storage\gcs.py:40:     def fetch_to_local(self, ref, dst=None, *, overwrite=False)
-> str:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-gdrive-api-
safety\backend\storage\gcs.py-41-         if dst is None:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-gdrive-api-
safety\backend\storage\gcs.py-42-             raise ValueError("GCSStorage.fetch_to_local
requires a dst path")
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-gdrive-api-
safety\backend\storage\gcs.py-43-         import os
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-gdrive-api-
safety\backend\storage\gcs.py-44-
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-

```

```

indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-gdrive-api-
safety\backend\storage\gcs.py-45-         dst = str(dst)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-gdrive-api-
safety\backend\storage\gcs.py-46-         if os.path.exists(dst) and not overwrite:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-gdrive-api-
safety\backend\storage\gcs.py-47-         return dst
...
</persisted-output>

```

21. user (2026-07-02T04:57:51.863Z)

```

40         def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
41             if dst is None:
42                 raise ValueError("GCStorage.fetch_to_local requires a dst path")
43             import os
44
45             dst = str(dst)
46             if os.path.exists(dst) and not overwrite:
47                 return dst
48             os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
49             self._blob(ref).download_to_filename(dst)
50             return dst
51
52         def put(self, local_path, ref, *, content_type=None) -> StorageRef:
53             self._blob(ref).upload_from_filename(str(local_path), content_type=content_type)
54             return ref
55
56         def exists(self, ref) -> bool:
57             return bool(self._blob(ref).exists())
58
59         def stat(self, ref) -> StorageStat:
60             b = self._blob(ref)
61             b.reload()
62             return StorageStat(
63                 size=int(b.size or 0),
64                 modified=b.updated,

```

22. user (2026-07-02T04:57:54.290Z)

```

run_all_tests.py
4:A single, discoverable entry point so you (or a reviewer / CI) can verify the
10: python run_all_tests.py --cov=backend --cov-report=term-missing # coverage
.github/workflows/ci.yml:145: run: python run_all_tests.py --cov=backend --cov=training
--cov-report=term --cov-fail-under=70
.github/workflows/nightly.yml:139: run: python run_all_tests.py --cov=backend
--cov=training --cov-report=term --cov-fail-under=70

```

23. user (2026-07-02T04:59:02.372Z)

```

ERROR tests/test_storage_gdrive_api.py::test_reput_updates_in_place_preserving_file_id
ERROR tests/test_storage_gdrive_api.py::test_put_failure_mid_upload_keeps_old_object_intact
ERROR tests/test_storage_gdrive_api.py::test_fetch_large_file_streams_in_chunks
ERROR tests/test_storage_gdrive_api.py::test_fetch_failure_mid_download_leaves_no_truncated_dst
ERROR tests/test_storage_gdrive_api.py::test_fetch_missing_raises - Attribute...
ERROR tests/test_storage_gdrive_api.py::test_fetch_requires_dst - AttributeEr...
ERROR tests/test_storage_gdrive_api.py::test_put_at_root_uses_root_folder - A...
5 passed, 9 errors in 0.70s

```

24. assistant (2026-07-02T04:59:03.095Z)

You've hit your session limit · resets 1:10pm (Asia/Calcutta)